

Trabajo Final Integrador

Catedra: Programación 2
Tecnicatura universitaria en
Programación

Alumnos: Andrés Fabre, Mariano Chirino,
Facundo Quiroga

17 de junio de 2026, Universidad Tecnológica
Nacional

Índice

Marco Teórico	3, 4, 5
• Java • UML • Persistencia con JDBC • DAO	3 4 4,5 5
Decisiones Técnicas y Arquitectura	6, 7, 8, 9, 10, 11
Dificultades y Decisiones	12
Bibliografía/Webgrafía	13
Links Principales	14

Marco teórico

Tecnologías utilizadas: java, UML, base de datos relacionales, persistencia con JDBC y DAO

Java

Java es un lenguaje de programación de propósito general y una plataforma informática. Su característica principal es que es multiplataforma, lo que significa que el código escrito puede ejecutarse en cualquier sistema operativo (como Windows, macOS o Linux) sin necesidad de ser modificado.

¿Cómo funciona?

- Escribe una vez, ejecuta en cualquier lugar: Esta es su filosofía central. El código no se comunica directamente con el sistema operativo, sino a través de una Máquina Virtual de Java (JVM). La JVM actúa como un "traductor" universal.
- Orientado a objetos: Organiza el software en "objetos" y "clases", lo que facilita la creación de programas complejos, reutilizables y seguros.
¿Para qué se utiliza?
- Aplicaciones empresariales: Es el lenguaje estándar utilizado por grandes bancos y corporaciones para gestionar datos y sistemas robustos.
- Aplicaciones Android: Es el lenguaje oficial (junto con Kotlin) utilizado para crear aplicaciones para teléfonos inteligentes y tablets.
- Servicios en la nube y Big Data: Muy solicitado en infraestructuras de servidores y procesamiento de datos a gran escala.

Base de datos relacionales

Una base de datos relacional es una colección de información que organiza datos en relaciones predefinidas, en la que los datos se almacenan en una o más tablas (o "relaciones") de columnas y filas, lo que facilita la visualización y la comprensión de cómo se relacionan las diferentes estructuras de datos entre sí. Las relaciones son conexiones lógicas entre las diferentes tablas y se establecen a partir de la interacción entre ellas.

UML

El Lenguaje Unificado de Modelado (UML) fue creado para forjar un lenguaje de modelado visual común y semántica y sintácticamente rico para la arquitectura, el diseño y la implementación de sistemas de software complejos, tanto en estructura como en comportamiento. UML tiene aplicaciones más allá del desarrollo de software, p. ej., en el flujo de procesos en la fabricación.

Es comparable a los planos usados en otros campos y consiste en diferentes tipos de diagramas. En general, los diagramas UML describen los límites, la estructura y el comportamiento del sistema y los objetos que contiene.

UML no es un lenguaje de programación, pero existen herramientas que se pueden usar para generar código en diversos lenguajes usando los diagramas UML. UML guarda una relación directa con el análisis y el diseño orientados a objetos.

Los lenguajes orientados a objetos dominan el mundo de la programación porque modelan los objetos del mundo real. UML es una combinación de varias notaciones orientadas a objetos: diseño orientado a objetos, técnica de modelado de objetos e ingeniería de software orientada a objetos.

Persistencia con JDBC

En Java, la persistencia consiste en guardar los datos de tu aplicación en un medio de almacenamiento no volátil (como una base de datos) para que no se pierdan al cerrar el programa.

JDBC (*Java Database Connectivity*) es la API oficial y de bajo nivel que permite a tu aplicación Java conectarse directamente a bases de datos relacionales, enviarle comandos SQL y recuperar o modificar información.

¿Cómo funciona la persistencia con JDBC?

El proceso para lograr la persistencia utilizando esta tecnología sigue un flujo estándar de bajo nivel:

1. Cargar el Driver: Se registra el controlador específico de la base de datos (por ejemplo, MySQL, PostgreSQL o Oracle) para que Java sepa cómo comunicarse con ella.
 2. Establecer la Conexión: Se utiliza la URL de la base de datos, el usuario y la contraseña para crear un puente de comunicación.
 3. Crear y Ejecutar Sentencias SQL: Se escriben consultas SQL directamente en código Java (comandos como SELECT, INSERT, UPDATE o DELETE) y se envían a la base de datos.
 4. Procesar Resultados: Los resultados de las consultas se leen fila por fila utilizando un objeto llamado ResultSet.
 5. Cerrar la Conexión: Se liberan los recursos de memoria y de red utilizados.
- Ventajas y Desventajas

A favor:

- Control total: Al escribir el código SQL directamente, puedes optimizar las consultas y realizar operaciones complejas a medida.
- Bajo consumo: Al no tener capas intermedias, puede ser más directo y rápido para tareas muy específicas.

En contra:

- Código repetitivo (Boilerplate): Requiere escribir mucho código solo para manejar conexiones y cerrar recursos.
- Dependencia de la base de datos: Si cambias de motor de base de datos, es posible que debas reescribir parte de tu código SQL.
- Complejidad: Traducir filas de una tabla de base de datos a objetos Java (y viceversa) manualmente es tedioso y propenso a errores.

DAO

En Java, DAO significa Data Access Object (Objeto de Acceso a Datos). Es un patrón de diseño que se utiliza para separar la lógica de negocios de tu aplicación de la lógica encargada de acceder a la base de datos o almacenamiento.

¿Por qué se utiliza?

- Separación de responsabilidades: Evita que escribas código de base de datos (SQL, conexiones, etc.) dentro de tus clases principales.
 - Mantenimiento: Si cambias de motor de base de datos (ej. de MySQL a Oracle), solo necesitas modificar las clases DAO sin afectar el resto de tu código.
 - Facilidad de pruebas: Permite probar tu lógica de negocio de manera aislada.
- ¿Cómo funciona en un sistema?

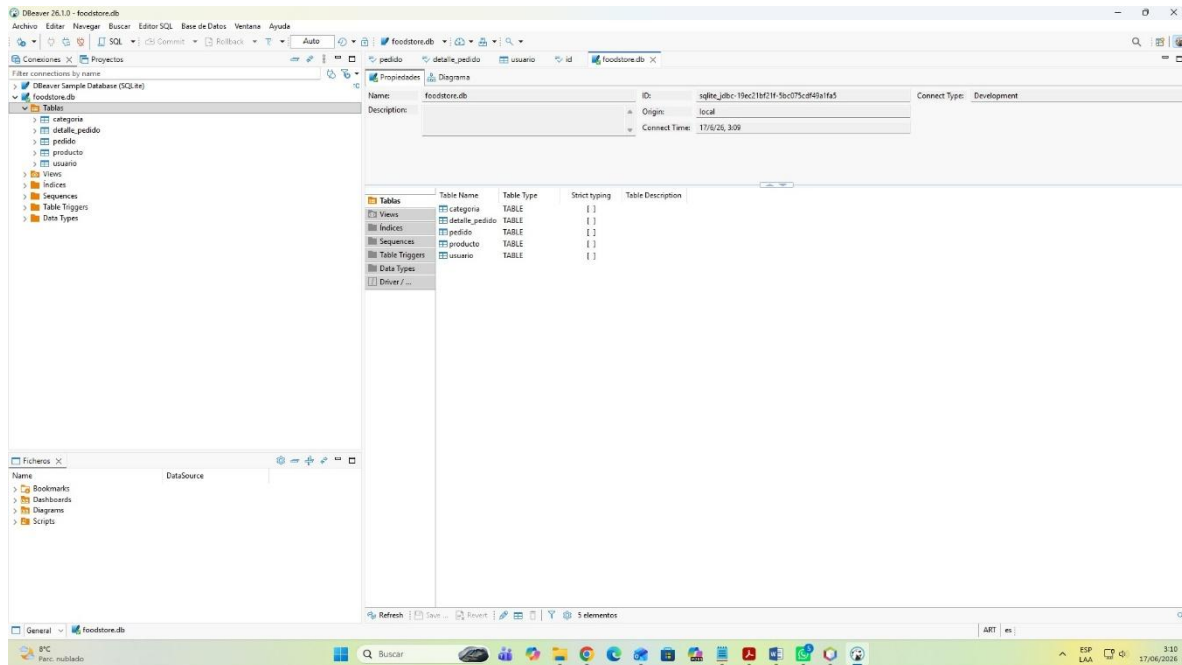
Un flujo típico de la arquitectura involucra tres elementos principales:

1. Modelo o Entidad (o DTO): Representa los datos (por ejemplo, una clase Usuario).
2. DAO: Es la clase o interfaz que contiene los métodos para realizar operaciones CRUD (Crear, Leer, Actualizar y Eliminar) sobre la entidad.
3. Capa de Negocio (Service): Utiliza el DAO para obtener o guardar información, pero no le importa *cómo* se conecta a la base de datos

Decisiones Técnicas y Arquitectura

La resolución del problema (Sistema de gestión de pedidos de comida) se resolvió de la siguiente manera:

- 1- Se estudió el modelo UML y las entidades
- 2- Se creó la base de datos y las tablas. En este caso se utilizó SQLite.



- 3- Elegimos el IDE para la programación del sistema, en este caso se prefirió netbeans.
- 4- Creamos el proyecto y los paquetes.

La arquitectura elegida fue la siguiente:

src/

└─ integrado/prog2/

└─ Main.java

└─ config/ (ConexionDB, lectura de configuración)

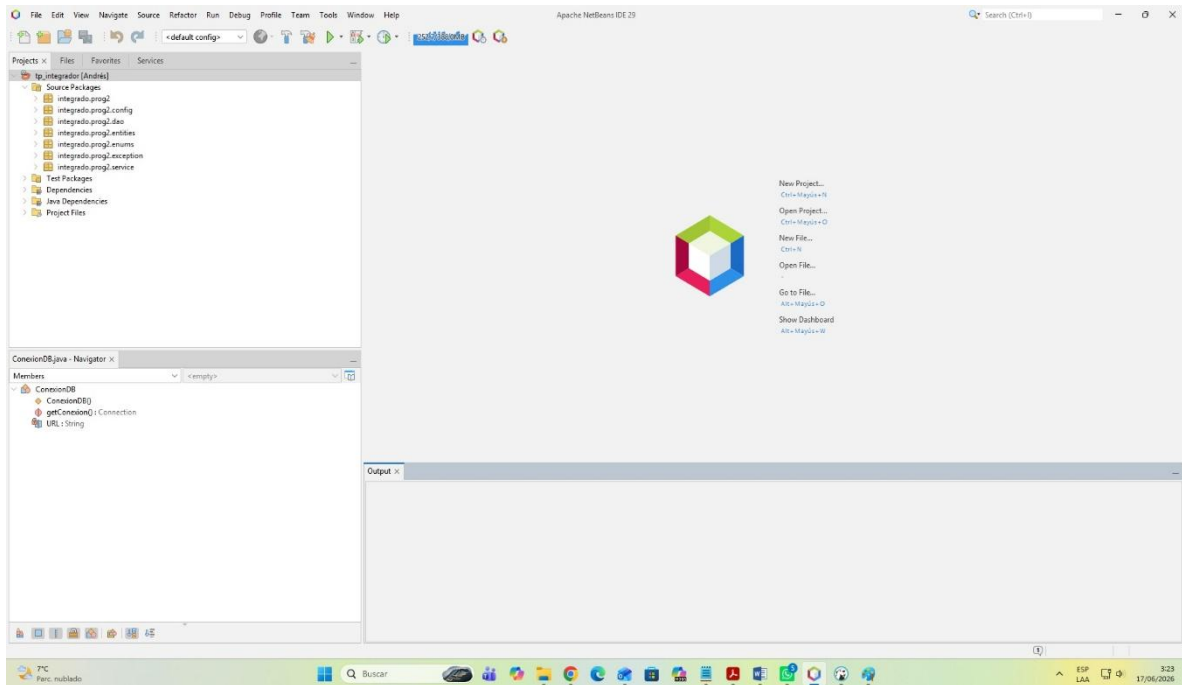
└─ entities/ (clases del UML)

└─ enums/ (Rol, Estado, FormaPago)

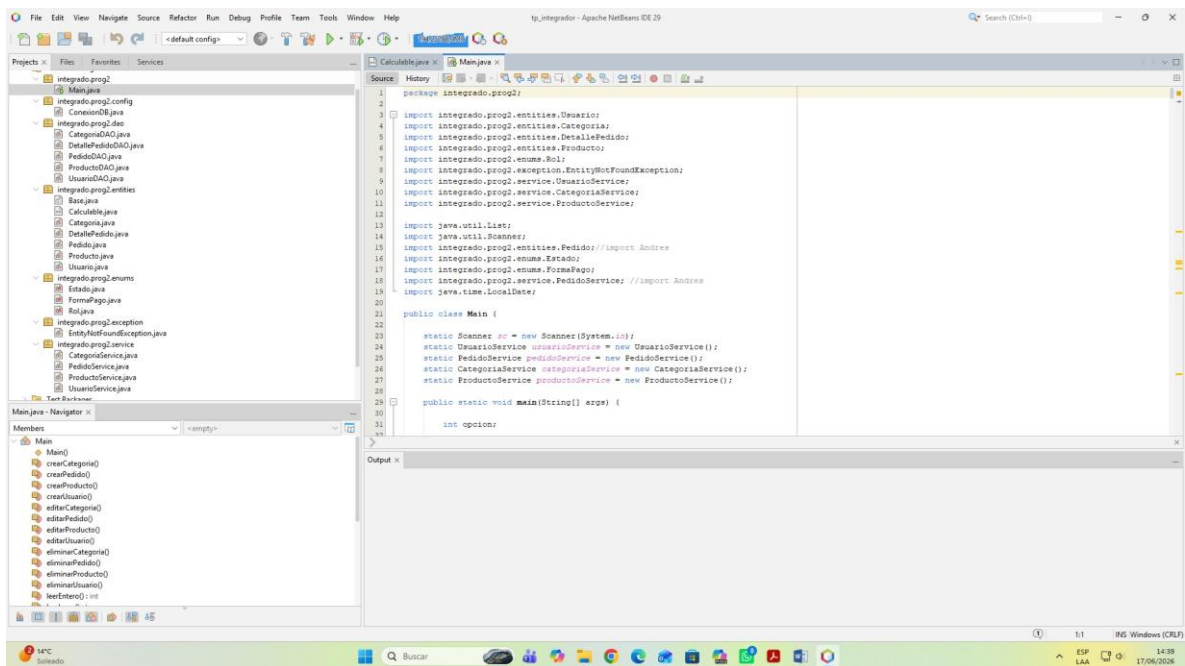
└─ dao/ (DAO por entidad + interfaces)

└─ service/ (servicios que usan DAO)

└─ exception/ (excepciones propias)

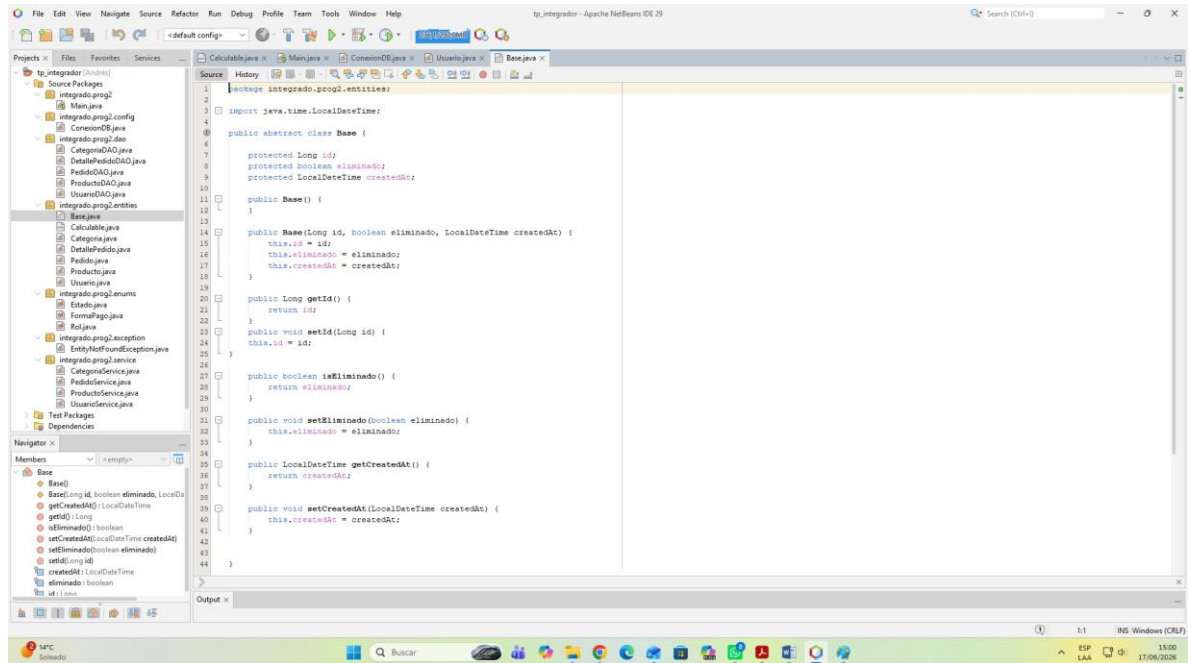


5- Creación de las clases, los enums y las interfaces, todos en sus respectivos paquetes. Se respetó el patrón de diseño DAO. Las entidades se crearon en el paquete “entities”, las interfaces en paquete “dao”, y la implementación en el paquete “service”. La clase encargada de hacer la conexión con la base de datos se encuentra en el paquete “config”.

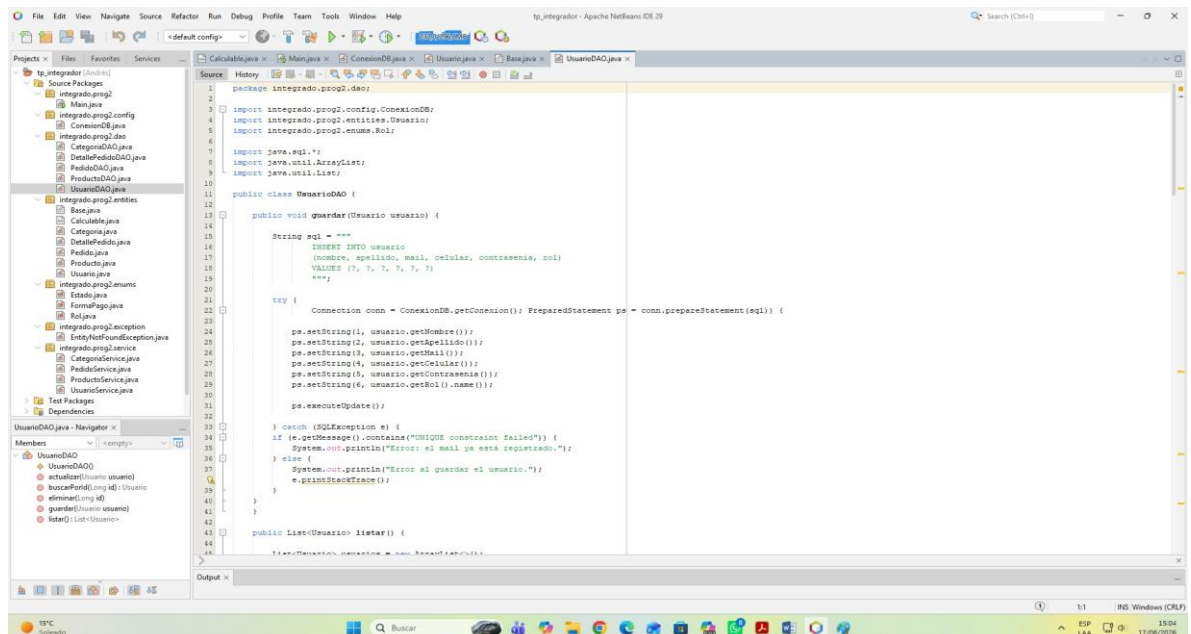


6- Se codifico el sistema en el lenguaje JAVA con JDBC SQLite.

- Primero se codificaron las entidades y la conexión con la base de datos, así como las excepciones. La primera clase en construirse fue la clase Base.java, posteriormente la clase Usuarios.java. Podemos observar que el modelo de dominio de la aplicación es completamente independiente de la base de datos o de la capa de persistencia (DAO).



- Después se hicieron las clases DAO y los servicios. La interfaz DAO define una API abstracta que realiza las operaciones CRUD



- Finalmente se codificó la clase Main la cual contiene principalmente los menús y las funciones de eliminar, editar, crear y listar objetos.

```

7 import integrado.prog2.enums.Rol;
8 import integrado.prog2.exception.EntityNotFoundException;
9 import integrado.prog2.service.UsuarioService;
10 import integrado.prog2.service.CategoriaService;
11 import integrado.prog2.service.ProductoService;
12
13 import java.util.List;
14 import java.util.Scanner;
15 import integrado.prog2.entities.Pedido; //import Andres
16 import integrado.prog2.enums.Estado;
17 import integrado.prog2.enums.FormaPago;
18 import integrado.prog2.service.PedidoService; //import Andres
19 import java.time.LocalDate;
20
21 public class Main {
22
23     static Scanner sc = new Scanner(System.in);
24     static UsuarioService usuarioService = new UsuarioService();
25     static PedidoService pedidoService = new PedidoService();
26     static CategoriaService categoriaService = new CategoriaService();
27     static ProductoService productoService = new ProductoService();
28
29     public static void main(String[] args) {
30
31         int opcion;
32
33         do {
34             System.out.println("\n=== FOOD STORE ===");
35             System.out.println("1. Categorias");
36             System.out.println("2. Productos");
37             System.out.println("3. Usuarios");
38             System.out.println("4. Pedidos");
39             System.out.println("0. Salir");
40             System.out.print("Seleccione: ");
41
42             opcion = LeerEntero();
43
44             switch (opcion) {
45                 case 1 -> menuCategorias();
46                 case 2 -> menuProductos();
47                 case 3 -> menuUsuarios();
48                 case 4 -> menuPedidos(); //Andres
49                 case 0 -> System.out.println("Saliendo...");
49                 default -> System.out.println("Opcion invalida.");
50             }
51         } while (opcion != 0);
52     }
53
54     //Metodos para crear, editar, eliminar y listar
55     public void crearCategoria() {
56         //Codigo para crear categoria
57     }
58     public void crearProducto() {
59         //Codigo para crear producto
60     }
61     public void crearUsuario() {
62         //Codigo para crear usuario
63     }
64     public void editarCategoria() {
65         //Codigo para editar categoria
66     }
67     public void editarProducto() {
68         //Codigo para editar producto
69     }
70     public void editarUsuario() {
71         //Codigo para editar usuario
72     }
73     public void eliminarCategoria() {
74         //Codigo para eliminar categoria
75     }
76     public void eliminarProducto() {
77         //Codigo para eliminar producto
78     }
79     public void eliminarUsuario() {
80         //Codigo para eliminar usuario
81     }
82     public void listarCategorias() {
83         //Codigo para listar categorias
84     }
85     public void listarProductos() {
86         //Codigo para listar productos
87     }
88     public void listarUsuarios() {
89         //Codigo para listar usuarios
90     }
91     public void listarPedidos() {
92         //Codigo para listar pedidos
93     }
94 }

```

7- Una vez terminado el código, se probó el sistema varias veces.

- Menú principal

```

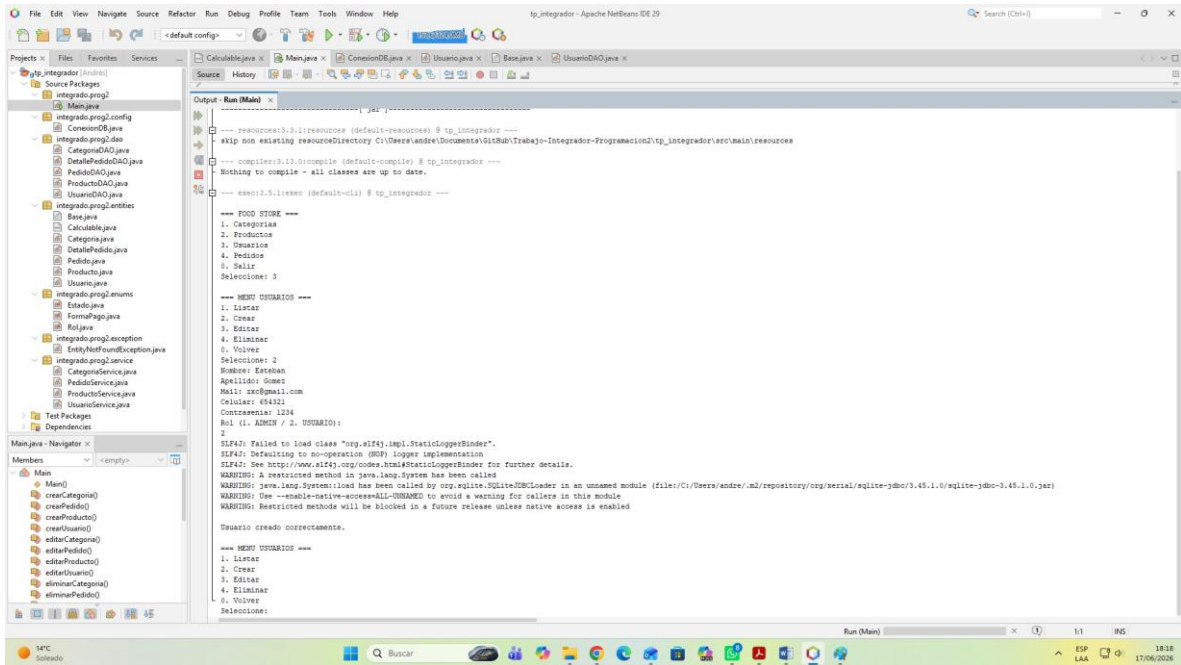
Output - Run (Main)
C:\Users\andrea\Documents\Github\Trabajo-Integrador-Programacion\tp_integrador> JAVA_HOME=C:\Program Files\Apache NetBeans\jdk\bin cmd /c "C:\Program Files\Apache NetBeans\java\bin\java.exe" -Denvr.makings
Scanning for projects...

===== [ JAR ] =====
===== resources\3.1\resources (default-resources) @ tp_integrador =====
skip non existing resourceDirectory C:\Users\andrea\Documents\Github\Trabajo-Integrador-Programacion\tp_integrador\src\main\resources
===== compiler\3.1\3\compile (default-compile) @ tp_integrador =====
Nothing to compile - all classes are up to date.
===== exec\3.1\exec (default-cli) @ tp_integrador =====

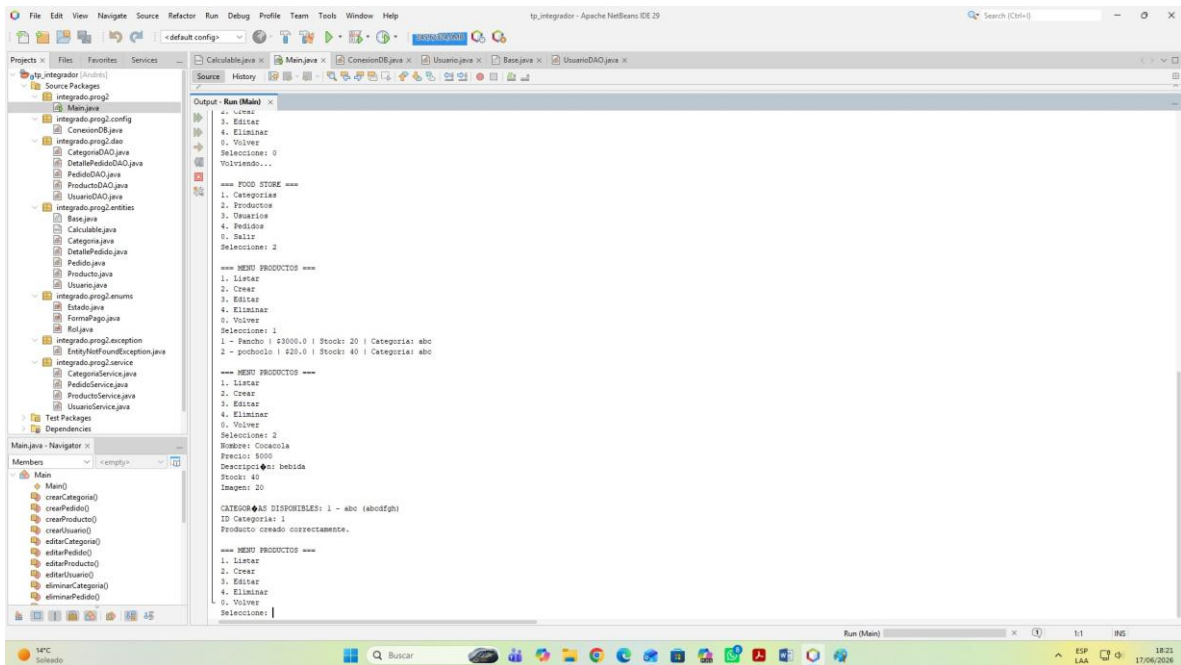
=== FOOD STORE ===
1. Categorias
2. Productos
3. Usuarios
4. Pedidos
0. Salir
Seleccione:

```

- Creación de un usuario



- Creación de un producto



- Creación de un pedido

```

1. Listar
2. Crear
3. Editar
4. Eliminar
5. Volver
Selecciones: 2

===== USUARIOS =====
Usuario(id=1, nombre="Andrés", apellido="Félix", mail="andf@outmail.com", rol=ADMIN)
Usuario(id=4, nombre="Alejandro", apellido="Puga", mail="alejandro@gmail.com", rol=USUARIO)
Usuario(id=7, nombre="Esteban", apellido="Gómez", mail="esteb@gmail.com", rol=USUARIO)
Ingresar ID usuario: 7

Forma de pago:
1. EFECTIVO
2. TARJETA
3. TRANSFERENCIA
Selecciones: 1

===== PRODUCTOS =====
1 - Pancho | $3000.0 | Stock: 20 | Categoría: abo
2 - prodcción | $20.0 | Stock: 40 | Categoría: abo
3 - CocaCola | $5000.0 | Stock: 40 | Categoría: abo
ID productos: 1
Cantidad: 3
Agregar otro producto? (S/N): S

===== PRODUCTOS =====
1 - Pancho | $3000.0 | Stock: 20 | Categoría: abo
2 - prodcción | $20.0 | Stock: 40 | Categoría: abo
3 - CocaCola | $5000.0 | Stock: 40 | Categoría: abo
ID productos: 3
Cantidad: 3
Agregar otro producto? (S/N): N
Pedido creado correctamente.
Total: 24000.0

===== MENU PEDIDOS =====
1. Listar
2. Crear
3. Editar
4. Eliminar
5. Volver
Selecciones: 1
Pedido(id=1, fecha=2024-06-15, estado=PENDIENTE, total=12000.0, formaPago=EFECTIVO)
Pedido(id=2, fecha=2024-06-15, estado=CONFIRMADO, total=9100.0, formaPago=TRANSFERENCIA)
Pedido(id=3, fecha=2024-06-17, estado=PENDIENTE, total=24000.0, formaPago=EFECTIVO)

===== MENU PEDIDOS =====
1. Listar
2. Crear
3. Editar
4. Eliminar
5. Volver
Selecciones:

```

- Editar un pedido

```

4. Listar
3. Editar
4. Eliminar
5. Volver
Selecciones: 1
Pedido(id=1, fecha=2024-06-15, estado=PENDIENTE, total=12000.0, formaPago=EFECTIVO)
Pedido(id=2, fecha=2024-06-15, estado=CONFIRMADO, total=9100.0, formaPago=TRANSFERENCIA)
Pedido(id=3, fecha=2024-06-17, estado=PENDIENTE, total=24000.0, formaPago=EFECTIVO)

===== MENU PEDIDOS =====
1. Listar
2. Crear
3. Editar
4. Eliminar
5. Volver
Selecciones: 3
Pedido(id=1, fecha=2024-06-15, estado=PENDIENTE, total=12000.0, formaPago=EFECTIVO)
Pedido(id=2, fecha=2024-06-15, estado=CONFIRMADO, total=9100.0, formaPago=TRANSFERENCIA)
Pedido(id=3, fecha=2024-06-17, estado=PENDIENTE, total=24000.0, formaPago=EFECTIVO)
Ingresar ID del pedido: 3
1. PENDIENTE
2. CONFIRMADO
3. TERMINADO
4. CANCELADO
Nuevo estado: 3
1. TRANSFERENCIA
2. TRANSFERENCIA
3. EFECTIVO
Forma de pago: 2
Pedido actualizado.

===== MENU PEDIDOS =====
1. Listar
2. Crear
3. Editar
4. Eliminar
5. Volver
Selecciones: 1
Pedido(id=1, fecha=2024-06-15, estado=PENDIENTE, total=12000.0, formaPago=EFECTIVO)
Pedido(id=2, fecha=2024-06-15, estado=CONFIRMADO, total=9100.0, formaPago=TRANSFERENCIA)
Pedido(id=3, fecha=2024-06-17, estado=TERMINADO, total=24000.0, formaPago=TRANSFERENCIA)

===== MENU PEDIDOS =====
1. Listar
2. Crear
3. Editar
4. Eliminar
5. Volver
Selecciones:

```

Dificultades y decisiones

- 1- La primera dificultad que afrontamos fue la asignación de tareas. Este trabajo integrador se llevó a cabo por 3 personas. Decidimos dividir el trabajo en 3 partes. Creamos un repositorio principal en GitHub, después hicimos 3 ramas personales más una rama Main, en donde volcar todas las actualizaciones/avances de cada integrante (merge).
 - El integrante A se encargaría de Categorías y Productos, entidades Categoría y Producto, CategoriaDAO, ProductoDAO, CategoriaService, ProductoService, Submenús de consola de ambos, Enum, Estado y FormaPag.
 - El integrante B realizaría la base de datos, Clase usuario, usuarioDAO, usuarioService, y las excepciones.
 - El integrante C haría Pedidos + Documentación, Entidades Pedido, DetallePedido, Interfaz Calculable, PedidoDAO, PedidoService, Submenú de Pedidos con transacciones, Excepciones propias, (EntityNotFoundException) y PDF académico completo.
- 2- La segunda dificultad que tuvimos fue en cuanto a la conexión con la base de datos. En un principio se implementó MySQL, pero tuvimos problemas de conexión/errores, entonces decidimos utilizar en su lugar SQLite, un sistema de gestión de base de datos relacional ligero, de código abierto y sin servidor. Para gestionarlo utilizamos el programa DBeaver en su última versión.
- 3- En tercer lugar, surgió el problema de no conocer el patrón de diseño DAO. Lo solucionamos accediendo a distintas fuentes oficial de internet, para poder desarrollar el sistema con este modelo que separa la lógica de negocio con la lógica de acceso a datos.
- 4- Finalmente, no tuvimos problemas a considerar con la codificación del sistema, utilizamos los conocimientos de java adquiridos en la catedra programación 2 más distintas fuentes oficiales de internet.

Bibliografía/webgrafía

- Complemento Práctico JDBC: Lo que no sale en los libros (libro)

<https://drive.google.com/drive/u/0/folders/1CCnCWyxchFUYkcOQNw9vp1hklD4pRi>

- Introducción a JDBC (Video)

<https://www.youtube.com/watch?v=StqvFxxoxwU>

- Configuración de un entorno JDBC (video)

<https://www.youtube.com/watch?v=rLKIO66iC74>

- Conexión Java +MySQL con JDBC (video)

https://www.youtube.com/watch?v=BzVkX_67BFQ

- SQLite tutorial (web)

<https://www.sqlitetutorial.net/sqlite-java/sqlite-jdbc-driver/>

- Conexión básica a SQLite con java (video)

https://www.youtube.com/watch?v=d-_Z6qfoLOk

- Complemento Práctico CRUD y Transacciones: El Control Tota (libro)

<https://drive.google.com/drive/u/0/folders/1smNIYbA7eZLEdObcgUNbCZtDjN-luhbj>

- Operaciones CRUD con JDBC (video)

<https://www.youtube.com/watch?v=Z68hbGWiflc>

- Transacciones con JDBC (video)

<https://www.youtube.com/watch?v=LVkU33ate5M>

- El patrón DAO en java (web)

<https://www.baeldung.com/java-dao-pattern>

- Objeto de acceso a datos (DAO) (web)

<https://www.oracle.com/java/technologies/data-access-object.html>

- Complemento Práctico: Dejando el Código Espagueti (Apunte)

<https://drive.google.com/drive/u/0/folders/1tn7vPYqDkXm0A2nZLRHz2NQc2MvUMfrY>

- JDBC con buenas practicas (video)

<https://www.youtube.com/watch?v=QP2bQG5M0Ng>

- Proyecto JDBC aplicando buenas practicas (video)

<https://www.youtube.com/watch?v=xiapCXf4ioU>

- Fundamentos de JAVA (tercera edición) Herbert Schildt

<https://maycolmo.github.io/pe/java.pdf>

- Aprende a programar con JAVA, un enfoque práctico partiendo de cero (2da edición) por Alfonso Jimenez Marin – Francisco Manuel Pérez Monto

<https://bibliotecadigital.utn.edu.ec/files/original/e66e408ffacdfefc16d90f02fdce6b8b94a74ab7.pdf>

Links Principales

- Repositorio del trabajo final integrador en GitHub

<https://github.com/marianoemir/Trabajo-Integrador-Programacion2>

- Video Explicativo